13 A GUIDED EXAMPLE

This chapter applies many of the lessons taught throughout this book to the construction of an example software solution. It goes from initial problem description through to testing the finished product. The solution it presents is simplified and intended to highlight certain concepts. Opportunities for making it more sophisticated are discussed at the end of the chapter.

PROBLEM DEFINITION

Design a computer-controlled home automation system. The system should control the following parts of the house.

Ventilation

This regulates moisture content in the air. Moisture levels should never exceed 70 per cent. Furthermore, the ventilation regularly supplies outdoor air into the house. To do this, ventilation should run regularly at programmed times.

Heating

Radiators can be turned on or off to regulate the house temperature. Each room has one radiator. A pleasant temperature is around 22 degrees. The temperature of a room should not fall below 18 degrees.

Lighting

The lighting in each room should be sensitive to whether the room is occupied or not. When the room is occupied, the lights should be on. If a person leaves the room, the lights should be turned off.

Control panel

The automation system should be capable of being centrally controlled via a control panel. Functions of the control panel:

- The ventilation system can be programmed to come on at certain times of day.
- The heating system's lowest and optimum temperatures can be configured.

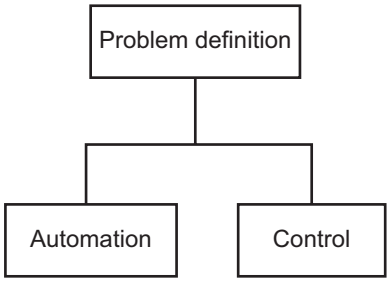
DISCLAIMER: do not take any material herein as advice on how to build proper automation systems.

PROBLEM DECOMPOSITION

I'll begin by decomposing the problem layer by layer into smaller addressable pieces.

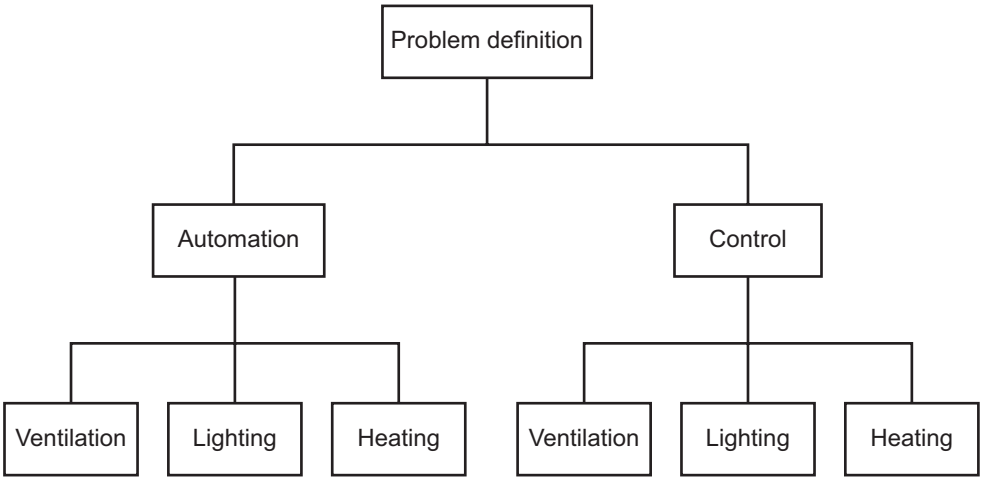
Examination of the problem reveals its essence concerns automation and user control. Each subsystem in the proposed system either carries out actions automatically or allows the user to control/configure it. The leads to the first stage in the breakdown (Figure 13.1).

Figure 13.1 Decomposition stage 1



In both cases, these actions concern all three major subsystems: ventilation, lighting and heating (Figure 13.2).

Figure 13.2 Decomposition stage 2



At the next level, I identify what the action means when applied to the subsystem in each case. For example, automatic behaviour of the lighting subsystem includes:

- measuring occupancy;
- reacting to a change in occupancy.

The resultant breakdown is in Figure 13.3.

At this point, I believe the sub-problems at the leaves of the tree are solvable individually.

FINDING PATTERNS

Next, I'll look at the problem description and the decomposition to identify any suitable patterns I could exploit.

Entities

Entities in the system are:

- rooms;
- central control panel;
- subsystems, that is:
  - heating;
  - lighting;
  - ventilation.
- components, that is:
  - radiators;
  - lights;
  - ventilator.
- sensors,[70] that is:
  - temperature sensor;
  - occupancy sensor;
  - moisture sensor.

Clearly, many distinct entities exist, but the majority of them can be combined into a more general concept, namely subsystems, components and sensors.

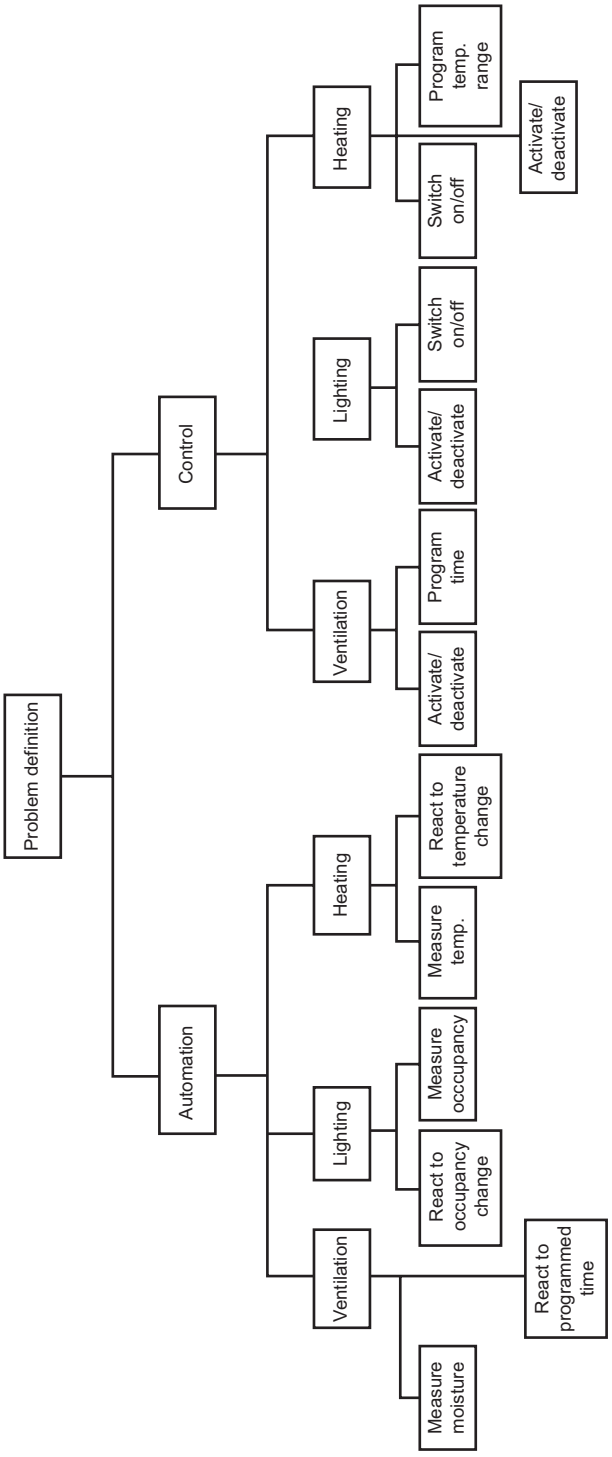


Figure 13.3 Decomposition stage 3

Actions

Actions in the system are:

- take measurements, that is:
  - temperature;
  - occupancy;
  - moisture level;
  - time.
- react to environmental change, that is:
  - switch radiators on/off;
  - switch lights on/off;
  - switch ventilator on/off.
- manually control the system, that is:
  - activate/deactivate an automation subsystem;
  - switch a component in a room on or off;
  - program ventilator time ranges;
  - program target temperature.

Like entities, most actions can be grouped into a smaller number of more general actions. You might have noticed two similar-sounding concepts in the list of actions: activating/deactivating and switching. Are they different names for the same thing? This exemplifies the need to have clear and well-defined terms.

In this case, I think they're distinct. **Activating/deactivating** applies to an automation subsystem (that is, moving it between automatic and manual modes), whereas **switching** means turning on or off a component in a room.

Properties

Properties in the system:

- environmental:
  - temperature;
  - occupied/unoccupied;
  - moist/dry.
- Mechanical:
  - subsystem active/inactive;
  - component on/off.

The system is rather a simplistic one, considering most properties have a binary nature. Hence, a room is either occupied or not, it's cold or not and so on. This greatly simplifies things.

Rules

Fortunately, this problem specification provides some relatively clear logical rules about how the solution should operate:

- If the temperature is less than 18 degrees, then radiator should activate.
- If the temperature is greater than the target temperature, then radiator should be inactive.
- If a room is occupied, then lights should be active otherwise lights should be inactive.
- If the moisture level is greater than 70 per cent **or** the current time is within the ventilator programmed time range, then the ventilator should be active, otherwise the ventilator should be inactive.

FORM GENERALISATIONS AND ABSTRACTIONS

As we look back over the decomposition, the following abstractions suggest themselves:

- The whole system consists of three **subsystems** and a **control panel**.
- Each subsystem consists of a collection of **sensors** and **components**.
- Each sensor is positioned in a **room**.
- A subsystem monitors sensors and reacts to their readings, ordering components to switch on/off according to **rules**.
- Rules map readings to **responses**.

Remarks

Some reasons why I judge certain abstractions to be valuable:

- Control panel: for the moment, we can refer to a control panel without deciding on its implementation. It could be a physical control panel or just as easily a smartphone app.
- Component: all components are essentially objects in rooms that can be switched on and off, so I can simplify the solution by considering one component entity instead of three different ones.
- Sensor: similarly, all sensors take readings, giving me one sensor entity instead of four.
- Rules: each subsystem has different logic driving its behaviour. However, the concept of a rule adequately encapsulates any piece of logic at the moment, so at this stage I don't have to worry about the intricacies of each one. Rules

also raise the possible need for something that carries out the monitoring and reacting in a subsystem, which we could call a **regulator**.

- Responses: a response, at the moment, is simply a switch on/off command, so this abstraction is probably overkill at this time. For simplicity's sake, I'll leave this abstraction out of the solution. However, it's not inconceivable to imagine that other components with gradated controls get added to the system in future, meaning that the nature of a response would vary depending on the type of component.

MODELS

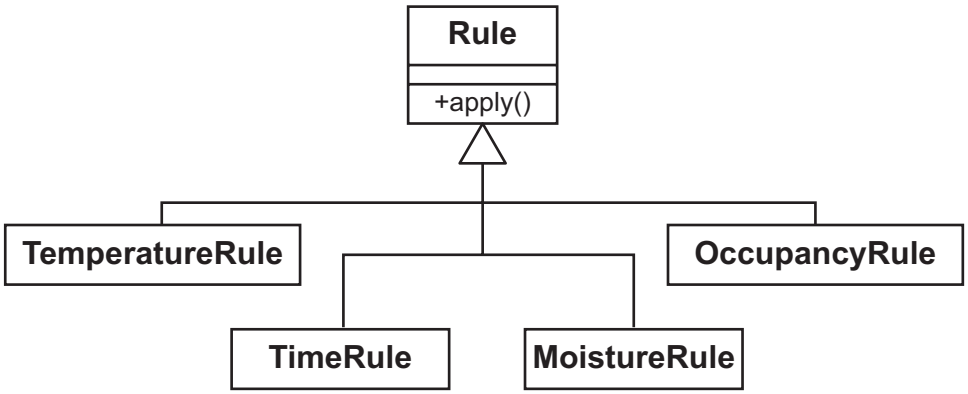
By extracting and analysing some generalised concepts, I've come to a tentative understanding of their behaviour. Next, I want to come to a better understanding of some of them, so I'll create models of those key parts.

Conceptual overview

By modelling the structure of the entities, I create a template for the objects that will eventually exist.

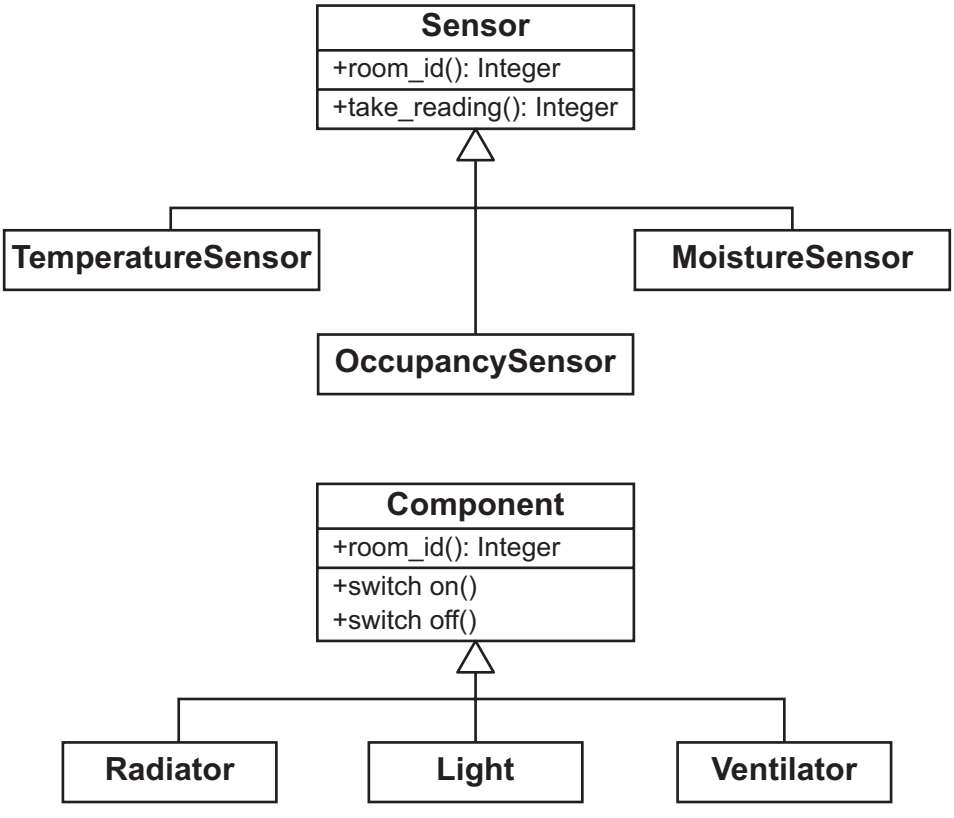
Figure 13.4 depicts the generalisation of rules. It's a fairly straightforward structure. In general, a rule can be applied (hence the `apply` function), and four specific types of rule exist.

Figure 13.4 Class diagram of rules



The story is very similar with components and sensors (Figure 13.5). A sensor can take readings and return the result. A component can be switched on or off. One notable thing is that the system can include multiple sensors and components. In order to control them individually, the solution needs to identify each uniquely, hence each possesses an ID number denoting which room it resides in.

Figure 13.5 Class diagrams of components and sensors



It seems now that the idea for a regulator was prescient. I can use one to bring all these concepts under its control (Figure 13.6). A regulator collects multiple sensors and components of a subsystem together and applies a rule when processing readings.

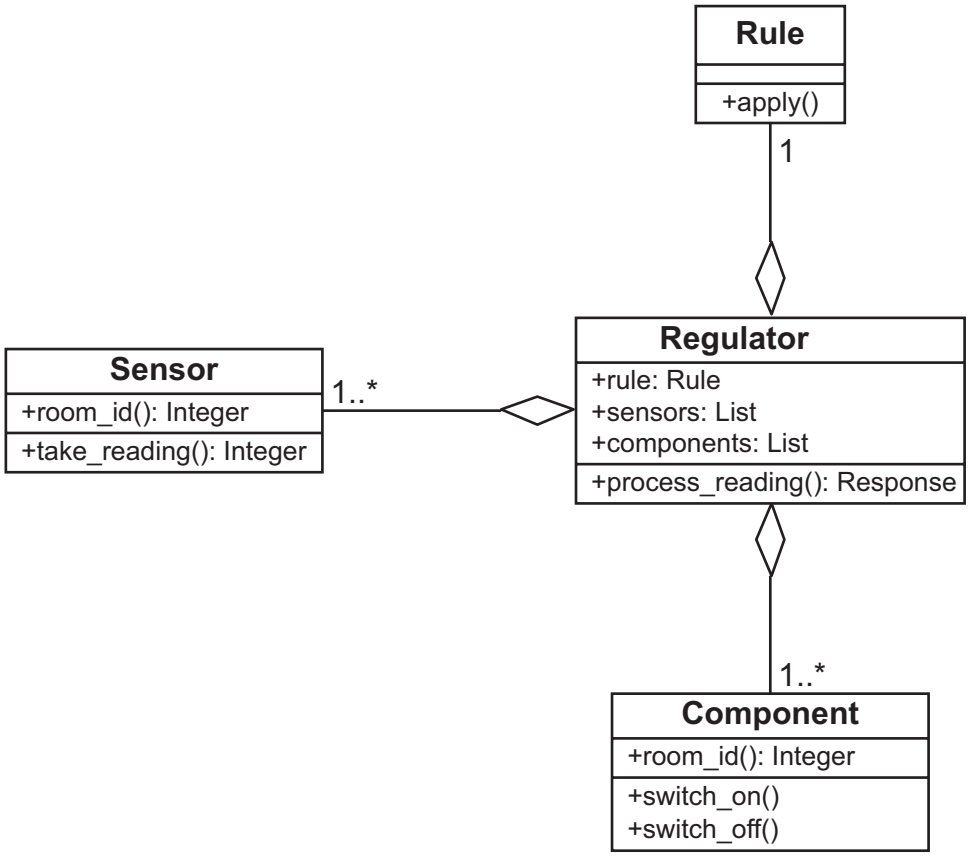
States and workflow

The system involves very careful control of multiple parts based on rules, so it's essential to understand that behaviour clearly, long before any code is written. Some workflows appear simple so I won't model everything, but I'll examine a few places that strike me as probable sources of complexity.

As a central controlling node in the solution, the regulator's behaviour involves a potentially complex interaction of several things. So, I'll create an activity diagram to model what a regulator does when it processes readings (Figure 13.7).

From the model, the regulator seems to cycle in a loop, taking measurements and switching components when necessary. One spot conceals some potential complexity, namely the activity where readings are mapped to a component state.

Figure 13.6 Class diagram of regulator



To explore this complexity, I could create state machine diagrams for each component that describe how state changes happen. Since space is limited, I'll just show one: the lighting.

Figure 13.8 shows what happens in terms of state alterations in the lighting subsystem. To switch the lighting on, the room must be occupied. Conversely, if the room subsequently becomes unoccupied, the system switches the lights back off.

This seems very simple. However, by modelling this process I forced myself to think it through, and this raised some practical questions. Specifically, I asked myself how occupancy might be measured. The industry standard for automatic lighting systems is through motion detection. The problem is, for a regulator to react promptly when someone enters the room, it would have to process sensor readings continuously (let's say once per second). But that would also mean the lights switch off whenever a room occupant happens to be motionless for a second or two. You can imagine how annoying that would be if you were sitting in a chair trying to watch TV.

Figure 13.7 Activity diagram of regulator

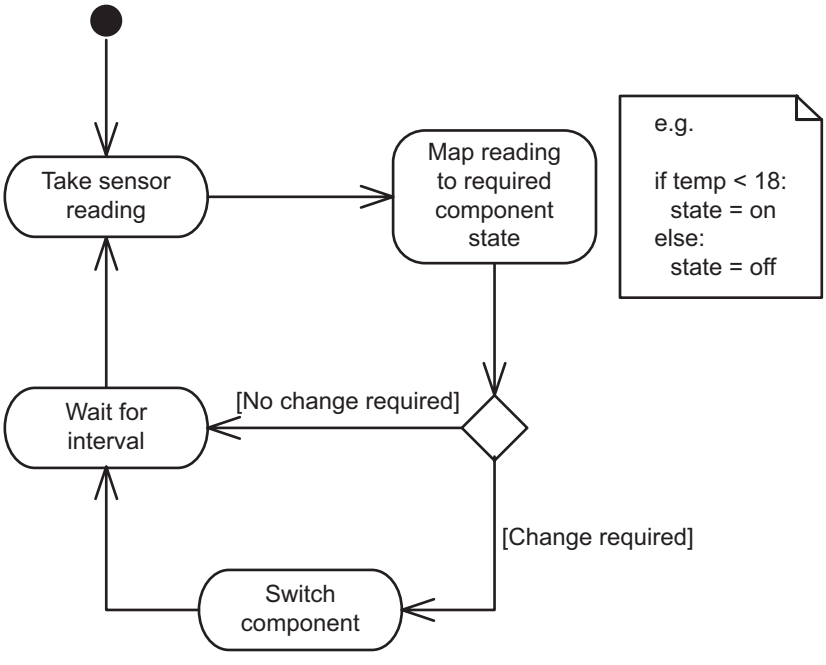
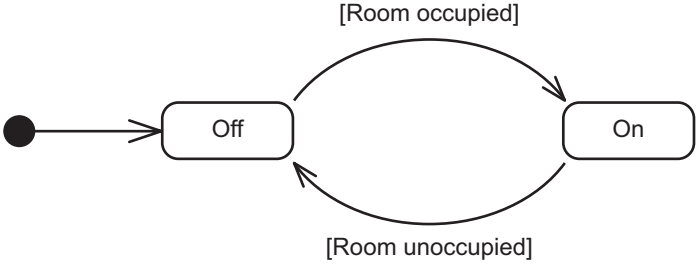
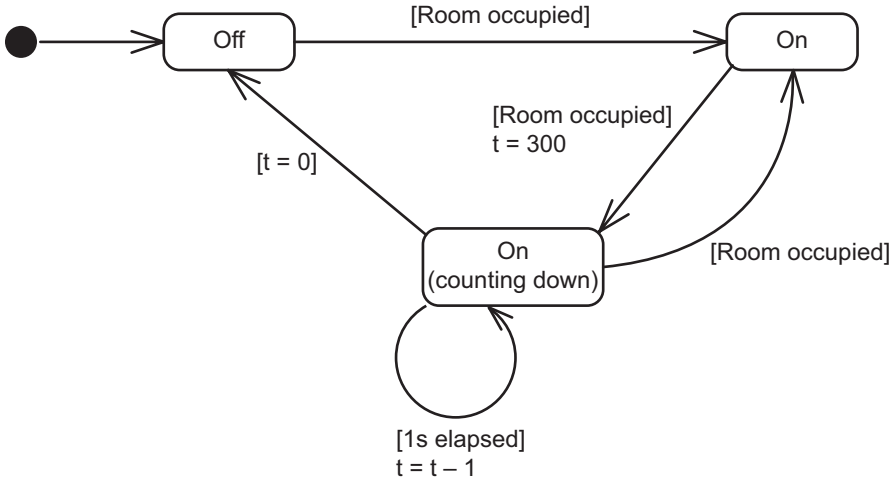


Figure 13.8 State diagram of applying a lighting rule



To take account of this, I've altered the rule for a lighting regulator by adding a time buffer (see Figure 13.9). Instead of switching the lights off immediately when no motion is detected, the regulator initiates a countdown. The lights will only be switched off when the countdown reaches zero. If, at any point, motion is again detected, the countdown is reset.

Figure 13.9 Revised state diagram of applying a lighting rule



ANNOTATED SOURCE CODE

Let's look through the code that implements the solution. First, the overall structure of the system:

```
- automation_system/
    - core/
        - __init__.py
        - parts.py
        - regulator.py
    - subsystems/
        - __init__.py
        - heating.py
        - lighting.py
        - ventilation.py
```

The system is divided into two packages:

1. core: the central parts of the home automation system.
2. subsystems: the different subsystems that make up a home automation system.

The tour begins with the core `parts` module. (Comments offer some description of what's happening in each module.)

```
# core/parts.py

# This module contains the basic parts of an automation subsystem.
# The intention is that the software developer uses these as the
```

```
# starting point for writing a new subsystem.

class Rule:

    def apply(self, reading, component):
        # The apply method takes a reading, applies the rule, and
        # responds by manipulating the component accordingly. It
        # should be overridden in a subclass.
        pass


class Component:

    def __init__(self, room_id):
        self.room_id = room_id
        # This attribute keeps track of whether the component is
        # off or not.
        self.off = True

    def switch_on(self):
        if self.off:
            self.off = False

    def switch_off(self):
        if not self.off:
            self.off = True


class Sensor:

    def __init__(self, room_id):
        self.room_id = room_id

    def take_reading(self):
        # Each sensor takes readings differently, so this
        # method should be overridden in the subclass.
        pass
```

These are abstract classes, not really intended as concrete parts, so let's turn our attention to how these core parts can be implemented as concrete subsystems. (We'll come back to the `core` package later.)

The simplest subsystem is heating:

```
# subsystems/heating.py

# Since this is a small class with few imports, there's
# little risk of imported names colliding, so we can
# use 'from <package> import <class>'
from core.parts import Component
```

```
from core.parts import Rule
from core.parts import Sensor

# The random package provides a means for generating random
# numbers. Why are we importing it? See below...
import random

class Radiator(Component):
    # The standard Component behaviour is fine, so we
    # don't need to override anything.
    pass


class TemperatureRule(Rule):

    def apply(self, reading, radiator):
        # This rule is quite simple. When it gets too cold,
        # the radiator comes on. It stays on until the room
        # reaches the pleasant temperature.
        if reading < 18:
            radiator.switch_on()
        elif reading > 22:
            radiator.switch_off()


class TemperatureSensor(Sensor):

    def take_reading(self):
        # This method gets the sensor reading. In reality,
        # it would be a rather complex piece of code that
        # interfaces directly with the hardware. For
        # demonstration purposes, it can just return a
        # random number between 16 and 25.
        return random.randint(16, 25)
```

The ventilation subsystem is not too much more taxing:

```
# subsystems/ventilation.py

# Ventilation behaviour partly depends on timing, so we'll
# use Python's built-in date and time library.
import datetime
import random

from core.parts import Component
from core.parts import Rule
from core.parts import Sensor
```

```
class Ventilator(Component):
    pass

class VentilationRule(Rule):
    # This class cheats a little and combines two
    # rules (moisture and time) into one.

    # Ventilation comes on between 17:00 and 20:00
    ventilation_on = datetime.time(17, 00)
    ventilation_off = datetime.time(20, 00)

    def apply(self, reading, ventilator):
        current_time = datetime.time(datetime.now())
        if reading > 70:
            # Golden rule: Ventilator must be on if moisture is
           # above 70%
            ventilator.switch_on()
        elif (current_time > ventilation_on and
                current_time < ventilation_off):
            # Even if moisture levels are fine, ventilation
           # must come on at allotted times
            ventilator.switch_on()
        else:
           # None of these conditions was met? Then switch
           # it off.
            ventilator.switch_off()

class MoistureSensor(Sensor):

    def take_reading(self):
        return random.randint(30, 80)
```

Finally, the lighting subsystem:

```
# subsystems/lighting.py

from core.parts import Component
from core.parts import Rule
from core.parts import Sensor

class Light(Component):

    def dim_off():
        pass
        # We could make the automation system more sophisticated
        # so that instead of switching a light straight off, it
        # dimmed the lights. If we did, the function would go
        # best here.
```

```
class LightingRule(Rule):
    # The lighting rule is a little more complex because it
    # incorporates a countdown.

    # This is the value assigned to the countdown when it's
    # reset
    # (300 s = 5 mins).
    time_limit = 300

    def __init__(self):
        # When the light subsystems is first started, the
        # countdown needs resetting.
        self._resetCountdown()

    def apply(self, reading, lights):
        if reading == True:
            # True means room is occupied.
            lights.switch_on()

            # We can now start the countdown.
            self._resetCountdown()
        else:
            # Room isn't occupied.
            if self.countdown == 0:
                # If the countdown has already reached zero,
                # switch off.
                lights.switch_off()
            else:
                 # Otherwise, we need to move the countdown
                 # along.
                self._tick()

    def _resetCountdown(self):
        # This method's name starts with an underscore as a
        # signal to Python meaning that this method is sort
        # of private and shouldn't concern whoever is using
        # this class.
        self.countdown = self.time_limit

    def _tick(self):
        # Count down one second (being careful not to count
        # lower than 0s)
        if self.countdown > 0:
            self.countdown = self.countdown - 1


class LightSensor(Sensor):

    def take_reading(self):
        return True if random.randint(0, 1) == 0 else
     False
```

That concludes the subsystems. Let's turn to the `core` package and look at the regulator:

```
class Regulator:

    def __init__(self, rule):
        # A regulator as specified is a generic type, i.e. it
        # doesn't become a specific type of regulator until
        # it is given a rule at run-time.
        # For example, if it is given a TemperatureRule, it
        # becomes a heating regulator.
        self.rule = rule

        # Sensors and components are stored in two
        # dictionaries. Each one maps a room number to a
        # specific sensor/component.
        self.sensors = dict()
        self.components = dict()

    def add_room(self, room_id, sensor, component):
        # Adds a room (along with its sensor and component)
        # to this regulator's purview.
        self.sensors[room_id] = sensor
        self.components[room_id] = component

    def process_reading(self):
        # Go through each sensor belonging to this regulator.
        for room_id, sensor in self.sensors.items():
            # For each one, take a reading and pass it
            # (along with the corresponding component) to this
            # regulator's rule. The rule can then use this
            # information to apply itself.

            try:
                # Of course, things can go wrong with the sensor
                # (it might be broken or communication might be
                # lost), so we need to be prepared for that.
                reading = sensor.take_reading()
            except SensorError:
                log.error('Cannot get sensor reading!', exc_info=True)

                self.rule.apply(reading, self.components[room_id])
```

TESTING

Let's look at a few illustrative test cases. I'll update the project layout to look like this:

```
- automation_system/
    - core/
        - __init__.py
        - parts.py
        - regulator.py
    - subsystems/
        - __init__.py
        - heating.py
        - lighting.py
        - ventilation.py
    - test/
        - test_heating.py
        - test_lighting.py
```

Unit tests

The unit tests will be recorded in files inside the `test` directory. First, unit testing the lighting rule. This tests only the `LightingRule` class.

```
# test_lighting.py
import unittest

import subsystems.lighting as light

class TestLightingRule(unittest.TestCase):

    def setUp(self):
        self.light_rule = light.LightingRule()
        self.light = light.Light(1)

    def test_lights_come_on(self):
        # Person enters the room
        self.light_rule.apply(True, self.light)

        # Light should be on
        self.assertFalse(self.light.off)

    def test_lights_stay_on(self):
        # Person enters the room
        self.light_rule.apply(True, self.light)

        # Person leaves the room
        self.light_rule.apply(False, self.light)
```

```
        # Light is still on
        self.assertFalse(self.light.off)

    def test_lights_countdown_to_off(self):
        # Person enters the room
        self.light_rule.apply(True, self.light)

        # Person leaves the room
        self.light_rule.apply(False, self.light)

        # Simulate 5 min countdown (i.e. 300 ticks)
        for _ in range(1, 300):
            self.light_rule.apply(False, self.light)

        # Bugs lurk in corners and congregate at boundaries:

        # Light should still be on
        self.assertFalse(self.light.off)

        # One more tick
        self.light_rule.apply(False, self.light)

        # Light should be off
        self.assertTrue(self.light.off)

if __name__ == '__main__':
    unittest.main()
```

The test run looks like this:

```
python -m unittest test/test_lighting.py
...
----------------------------------------------------------
Ran 3 tests in 0.000s

OK
```

System testing

Once individual units are shown to work well, we should put those units together to simulate use of the whole system. The following example integrates all parts of the heating subsystem and verifies that they work together properly.

```
# test/test_heating.py
import unittest

import core.regulator as regulator
import subsystems.heating as heat
```

```
def temp_low(self):
    return 17

def temp_ok(self):
    return 21

def temp_high(self):
    return 23

class TestHeatingSubsystem(unittest.TestCase):

    def setUp(self):
        # Fully simulate a regulator
        room_id = 1
        self.temperature_sensor = heat.TemperatureSensor(room_id)
        self.radiator = heat.Radiator(room_id)
        self.heat_regulator = regulator.Regulator(heat.TemperatureRule())
        self.heat_regulator.add_room(room_id, self.temperature_sensor,
self.radiator)

    def test_heating_off(self):
        # For testing purposes, we have to simulate sensor
        # outputs at different temperatures. The following line
        # replaces the take_reading method that a Radiator has
        # and replaces it with the temp_ok function
        # (defined above).
        heat.TemperatureSensor.take_reading = temp_ok

        # The temperature sensor will therefore return 21
        # degrees when the regulator processes a reading.
        self.heat_regulator.process_reading()

        # Radiator is off
        self.assertTrue(self.radiator.off)

    def test_heating_comes_on_and_stays_on(self):
        # The temperature sensor will now return 17
        # degrees when the regulator processes a reading.
        heat.TemperatureSensor.take_reading = temp_low
        self.heat_regulator.process_reading()

        # Radiator is on.
        self.assertFalse(self.radiator.off)

        # The temperature sensor will now return 21
        heat.TemperatureSensor.take_reading = temp_ok
        self.heat_regulator.process_reading()

        # Radiator is still on.
        self.assertFalse(self.radiator.off)
```

```
    def test_heating_on_then_off(self):
        heat.TemperatureSensor.take_reading = temp_low
        self.heat_regulator.process_reading()

        # Radiator is on.
        self.assertFalse(self.radiator.off)

        # Now temperature is too high
        heat.TemperatureSensor.take_reading = temp_high
        self.heat_regulator.process_reading()

        # Radiator is now off.
        self.assertTrue(self.radiator.off)

if __name__ == '__main__':
    unittest.main()
```

OPPORTUNITIES FOR IMPROVEMENT

As I said at the beginning, this is a very simplistic solution, optimised to show a few, easy-to-understand concepts. Many opportunities exist to make this example a more sophisticated and functional solution.

Here are a few I can think of:

- Add the code for the control panel.
- A regulator that supports multiple rules, so, for example, a ventilator can have separate rules for moisture and time.
- A component that has a more sophisticated status (such as dimmable lights).
- Add logging messages at appropriate places to aid debugging and monitoring.
- Write a system test that simulates three subsystems running at the same time.
- An improved countdown in the lighting subsystem, for example:
  - The lighting rule currently assumes that a regulator processes readings once per second. That may not be true in future and would break existing functionality if it changed (for example, a lighting regulator that processes two readings per second will run down the countdown after only 2.5 minutes). Alter the way the countdown measures time, so the LightingRule's countdown doesn't depend on the regulator.[71]
  - **Advanced:** make the countdown process a concurrent thread of execution.[72]

What improvements can you think of?

Consider it an exercise to take whichever ideas you prefer and turn this simple system into a great home automation solution of your own.